Timed Alignments

Thomas Chatain
LMF, ENS Paris-Saclay,
CNRS, Université Paris-Saclay, Inria
Gif-sur-Yvette, France
thomas.chatain@ens-paris-saclay.fr
ORCID 0000-0002-1470-5074

Neha Rino
LMF, ENS Paris-Saclay,
CNRS, Université Paris-Saclay, Inria
Gif-sur-Yvette, France
neha.rino@ens-paris-saclay.fr

Ngoc Long Tran
LMF, ENS Paris-Saclay, CentraleSupelec
CNRS, Université Paris-Saclay, Inria
Gif-sur-Yvette, France
ngoc-long.tran@student-cs.fr

***Abstract*—The subject of this paper is to study conformance checking for timed models, that is, process models that consider both the sequence of events in a process as well as the timestamps at which each event is recorded. Time-aware process mining is a growing subfield of research, and as tools that seek to discover timing related properties in processes develop, so does the need for conformance checking techniques that can tackle time constraints and provide insightful quality measures for time-aware process models. In particular, one of the most useful conformance artefacts is the alignment, that is, finding the minimal changes necessary to correct a new observation to conform to a process model. In this paper, we set our problem of timed alignment and solve two cases each corresponding to a different metric over time processes.**

***Index Terms*—Conformance checking, Alignments, Timestamps, Time Petri nets**

I. Introduction

Process mining studies vast systems through their event logs, and seeks to extract meaningful ways to model the underlying patterns or processes that govern the behaviour of the system in order to better understand it, or predict it's future behaviour [15] [7]. Once such a process model is obtained, through machine learning or otherwise, it is natural to ask how one is sure the obtained model is a reasonable approximation of the system's behaviour, especially given the lack of explainability in the domain of machine learning. Conformance checking is the art of judging the performance of a process model by relating the modelled behaviour and the observed behaviour of a process, without depending on the origin of the model [7]. Observed behaviour comes in the form of traces in an event log, a sequence of events that occur during the functioning of the system, while process models are blueprints that describe what the underlying process of the system is expected to look like. Measures of how well a model reflects system behaviour include fitness, precision, generalisation, and simplicity. We often do not want the model to precisely generate all possible future system behaviour, but neither should it simply regurgitate the event log and accept no new system behaviours. Hence, what is often sought is a process model that can, up to some small error factor, approximate any reasonable future system behaviour.

In Arya Adriansyah's seminal thesis [2], we obtain the notion of an alignment, that is, the minimal series of corrections needed to transform an observed trace into the execution of the process model that most closely mimics it. It is often seen as an execution of the synchronised product of the process model and a *trace model* i.e. a simple model that captures exactly one event log trace. This gives us a series of edits, usually insertions or deletions, that transform the observed trace into a process trace. Alignments thereby pinpoint exactly where inevitable deviations from expected behaviour occur, and the more distant the aligning word of a model is from its observed trace, the worse the model is at reflecting real system behaviour.

Process models can be represented using a variety of formal objects, including Petri nets. Assuming the event logs are a list of words over a finite alphabet (the set of possible discrete events), the problem of calculating the alignment has been extensively studied [2] [5]. The notion of distance used on these words used is usually either Hamming distance or Levenshtein's edit distance. It is natural to want to study explicitly timed systems, as by considering events along with their timestamps when mining processes, we can glean information about the minimum delay between two events, the maximum duration the system takes to converge upon a state, or check deadlines, all of which are highly relevant in real world applications [10] [12] [13]. In addition, one may want to predict the timestamps of processes [1]. Time-aware process mining seeks to study both what the underlying

Neha Rino was funded by the International Master's Scholarships Program IDEX of Université Paris-Saclay.

processes govern system behaviour are, and what time constraints they obey [14] [11] [16]. In the process mining community, there are ways to use existing process model notation in order to denote time constraints. BPMN 2.0 comes equipped with *timer events* and can record absolute, relative, and cyclical time constraints. For our purposes, we use time Petri nets, that is, Petri nets augmented with the ability to record and check the duration it takes to fire a transition once enabled, using which one can impose certain constraints on the relationships between the timestamps of different events.

As time-aware process mining grows popular, new quality measures and conformance checking techniques must be developed that are sensitive to temporal constraints, but so far in the study of alignments as a conformance checking artefact, we notice that the process model used is never time-aware. For this, one needs to define distance functions that can meaningfully compare and separate different time processes. This paper seeks to provide a framework by which to do the very same, and set and solve the alignment problem for time-aware processes.

In this paper, we start in Section III by formally setting the alignment problem and, proposing three different distance functions over timed words. In Section IV-B we look at aligning under d_t, and give a quadratic time alignment algorithm in the case of a structurally restricted class of models (sequential causal processes). For all other types of models, in Section IV-A we present an encoding as a linear programming problem, albeit with slower worst-case time complexity. Finally in section V we study d_θ, which utilises the structure of the process model in its calculations, and we present a straightforward algorithm for solving the alignment in this case, whose time complexity is linear in the size of the causal process and the transition set of the model. The second setting, and the notion of delay edits that we proposed, provide new insight into the nature of time processes and their properties.

The proofs of the lemmas and theorems are available in [9].

II. Preliminaries

We represent events as pairs (a, t) where $a \in \Sigma$ is the name of the action, and t denotes the time at which said action was taken.

Definition 1. *A timed trace is a sequence $\gamma \in (\Sigma \times \mathbb{R}^+)^*$ of timed events, seen as a timed word.*

We will often ignore the untimed parts of timed words, i.e., the projection onto Σ^*, leaving just the timestamps, a sequence belonging to $\mathbb{R}^{+*}$. The timed process model we use here is a labeled time Petri net.

Definition 2 (Labelled Time Petri Net)**.** *A labeled time Petri net (or TPN) is a tuple $N = (P, T, F, SI, \Sigma, \lambda, M_0, M_f)$, where P, T are disjoint sets of places and transitions respectively, $F \subseteq (P \times T) \cup (T \times P)$ is the flow relation, $SI : T \to \mathbb{Q}^+ \times (\mathbb{Q}^+ \cup \{\infty\})$ is the static interval function, where $SI(t) = (Eft(t), Lft(t))$ such that Eft stands for earliest firing time, and Lft for latest firing time, $\lambda : T \to \Sigma$ is the labeling function, labeling transitions with actions from the action set Σ, and $M_0, M_f : P \to \mathbb{N}$ are the initial and final markings.*

Given a transition $t \in T$ we define the pre-set of t as ${}^\bullet t = \{p \in P | (p, t) \in F\}$ and its post-set similarly is defined as $t^\bullet = \{p \in P | (t, p) \in F\}$, (the presets and postsets of places are defined similarly). A transition t of a TPN is *enabled* at marking M iff $\forall p \in {}^\bullet t : M(p) > 0$. The set of all enabled transitions at a marking M is denoted by *Enabled(M)*.

A *state* of a TPN $N = (P, T, F, SI, \Sigma, \lambda, M_0, M_f)$ is a pair $S = (M, I)$, where M is a marking of N and $I : Enabled(M) \to \mathbb{R}^+$ is called the *clock function*. The initial state is $(M_0, \mathbf{0})$, where $\mathbf{0}$ is the zero function.

A transition t can fire from state $S = (M, I)$ after delay $\theta \in \mathbb{R}^+$ iff t is enabled at M, and $I(t) + \theta \in [Eft(t), Lft(t)]$ and for every other t' enabled at M, $I(t') + \theta \leq Lft(t')$. This firing is denoted $(M, I)[t\rangle(M'', I')$ with the new state (M'', I') defined as follows:

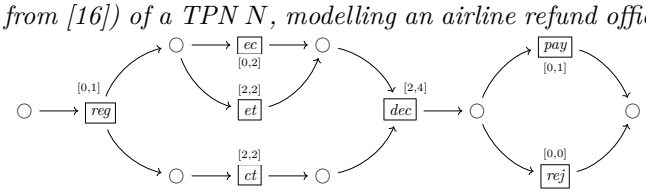
$$M'' = M' \cup t^\bullet \text{ where } M' = M \setminus {}^\bullet t$$

$$I'(t) = \begin{cases} I(t) + \theta & \text{If } t \in Enabled(M'') \cap Enabled(M') \\ 0 & \text{If } t \in Enabled(M'') \setminus Enabled(M') \end{cases}$$

A valid execution of the model begins at the initial marking, fires a sequence of transitions and reaches M_f, with any clock function I.

Definition 3 (Language $\mathcal{L}(N)$ of a TPN)**.** *A word $w = (a_0, a_1, \ldots a_n) \in \Sigma^*$ is in the language of the labeled TPN $\mathcal{L}(N)$ if there is a firable sequence of pairs, of transitions and their timestamps, $(t_0, t_1 \ldots t_n) \in (T, \mathbb{R}^+)^*$ such that $(\lambda(t_0), \lambda(t_1), \ldots, \lambda(t_n)) = w$ and they transform the initial marking into the final one, that is, for some clock function I on M_f, $(M_0, \mathbf{0})[t_0, t_1, \ldots t_n\rangle(M_f, I)$.*

Example 1. *Consider the following example (adapted from [16]) of a TPN N, modelling an airline refund office:*



The action labels are as follows : Register request (reg), Examine thoroughly (et), Examine casually (ec), Check ticket (ct), Decide (dec), Pay (pay), and Reject (rej).

The firing sequence $(reg, 1)(ec, 2)(ct, 3)(dec, 7)(rej, 7)$ is a valid execution of N. The initial marking only has reg enabled, and registering the request at time 1 updates the marking by removing the token from reg's preplace and filling its two post-places, thereby enabling $ec, et,$ and ct. Now ec fires at 2, satisfying the constraint, filling one preplace of dec. Then ct fires, having been enabled for 2

units of time. Now, finally, dec is enabled, and fires after 4 *units of having been enabled, and rej is fired immediately after.*

Now we will want to build some vocabulary to help us talk about individual executions of timed words on TPNs, that take the form of branching paths.

Definition 4 (Causal Net)**.** *A causal net* $CN = (B, E, G)$ *is a finitary, acyclic net where* $\forall b \in B : |b^\bullet| \leq 1 \wedge |{}^\bullet b| \leq 1$. B *(conditions) and* E *(events) are the vertices, and* G *denotes the edges.*

Hence, multiple elements of B or E map to the same element in P or T respectively, a notion that is expressed via the map p defined below.

Definition 5 (Homomorphism)**.** *Let* N *be a TPN with place set* P *and transition set* T*, and* $CN = (B, E, G)$ *be a causal net. A mapping* $p : B \cup E \to P \cup T$ *is a homomorphism if* $p(B) \subseteq P$*,* $p(E) \subseteq T$ *and* $\forall e \in E$ *the restriction of* p *to* ${}^\bullet e$ *is a bijection between* ${}^\bullet e$ *and* ${}^\bullet p(e)$ *and the restriction of* p *to* $e^\bullet$ *is a bijection between* $e^\bullet$ *and* $p(e)^\bullet$ *and the restriction of* p *to* $Min(CN)$ *is a bijection between* $Min(CN)$ *and* M_0*, where* $Min(CN)$ *is the set of vertices of* CN *that are minimal under the* G *pre-ordering.*

Definition 6 (Causal Process)**.** *A causal process of a TPN is a pair* (CN, p) *where* CN *is a causal net and* p *is a homomorphism from* CN *to* TPN*.*

Using p, elements of CN are identified with their corresponding *originals* in the TPN, and as a result, any causal process corresponds uniquely to an untimed run on the untimed version of a given TPN.

Definition 7 (Timing Function)**.** *A timing function* $\tau : E \to \mathbb{R}$ *is a function from events of a causal process into time values.*

Now, in order to study *timed* executions, we want to be able to add a timing function to the causal process we defined above, thereby allowing us to record when different transitions are taken. The following definitions, properties and theorems in this subsection cover the material developed in Aura and Lilius' article on Time Processes [3].

Given the definition of the timing function, we try to see how a time process unfolds. First the initial events, that is, those enabled at $Min(CN)$ happen, and as each event occurs, new events are enabled and disabled. In order to keep track of this, we first define the auxiliary Cut function that represents the effect of having all the events in a subset fire simultaneously (we only use Cut on sets of events that can concurrently fire), as defined below.

$$Cut(E') = (E'^\bullet \cup Min(NS)) \setminus {}^\bullet E'.$$

Now we can define the *time of enabling* for a transition t of a TPN in a set of conditions B' of its causal process as $TOE(B', t) =$

$$max(\{\tau({}^\bullet b) | b \in B' \setminus Min(CN) \wedge p(b) \in {}^\bullet t\} \cup \{0\})$$

We must now verify that this timing function does indeed represent a valid execution that obeys all the static interval constraints, as below :

Definition 8 (Valid Timing)**.** *A timing function* τ *is a valid timing of the causal process iff*

$$\forall e \in E : \tau(e) \geq TOE({}^\bullet e, p(e)) + Eft(p(e))$$

$$\forall e \in E : \forall t \in Enabled(p(C_e)) : \tau(e) \leq TOE(C_e, t) + Lft(t)$$

where $C_e = Cut(Earlier(e))$*.*

Here, checking every element of C_e might seem unnecessary but in certain nets, complex dependencies between transitions can cause causally unrelated transitions to force transitions to fire or be disabled, due to urgency. This phenomenon is known as confusion, and hence to guard against this, a timing function must check that it never leaves a section of the process behind, completing the firing or disabling of every event in $Earlier(e)$ before it can reason about the fireability of e.

Definition 9 (Time Process)**.** *A time process of a TPN* N *is a triple* (CN, p, τ) *where* τ *is a valid timing of* (CN, p) *which is a causal process of* N*.*

A few last notions here will assist us when we consider a simpler class of TPNs, that allow for much easier validity checking.

Definition 10 (Extended Free Choice)**.** *A TPN is extended free choice iff for all two transitions* t *and* t'*,* ${}^\bullet t \cap {}^\bullet t' \neq \emptyset$ *implies* ${}^\bullet t = {}^\bullet t'$*.*

This class of TPNs ensure that the net is *confusion-free*, that is, causally unrelated events cannot affect the fireability of other events. This means that in order to check for the validity of a timing function, checking all of C_e as before is no longer necessary.

How do we construct a valid timing function on the fly? Given a partial time process of a TPN, we want to be able to study the effect of firing particular transitions amongst the set of transitions currently enabled. In order to do so, we notice the following class of transitions.

A transition t is a *choice* at $B' \subseteq B$ iff B' is a co-set that maps injectively to places and ${}^\bullet t = p(B')$, and a choice is an *extension* of the process iff $B' \subseteq Cut(E)$.

These transitions reflect exactly the transitions enabled at a particular moment in the evolution of the process. Either they must be fired or disabled, by the urgency condition.

Given the above, Aura and Lilius characterise a method by which one can build partial time processes inductively, building forward while maintaining the validity of the timing function, denoted as keeping the process complete with respect to the timing function :

Definition 11. *A causal process* (CN, p) *of a TPN where* $CN = (B, E, G)$ *is said to be complete with respect to a*

timing function τ iff for every extension transition t of the process,

$$\max\{\tau(e)|e \in E\} \leq TOE(Cut(E), t) + Lft(t).$$

Now given this, the following theorem due to Aura and Lilius [3] holds :

Theorem 1. *Let N be an extended free choice TPN and (CN, p) a causal process of N, where $CN = (B, E, G)$. A timing function τ is valid iff the following criteria hold :*

1) $\forall e \in E : Eft(p(e)) \leq \tau(e) - TOE({}^\bullet e, p(e))$
2) $\tau(e) - TOE({}^\bullet e, p(e)) \leq \min\{Lft(t)|\, {}^\bullet t = {}^\bullet p(e)\}$
3) *(CN, p) is complete with respect to τ, i.e, for every extension transition t of the process,*

$$\max\{\tau(e)|e \in E\} \leq TOE(Cut(E), t) + Lft(t)$$

III. The Alignment Problem in Timed Settings

The alignment problem, given a trace from an event log and a process model, involves finding a valid execution of the model that is closest to the trace under some metric.

Definition 12 (The General Alignment Problem)**.** *Given a process model N denoted by a TPN and a timed trace σ we wish to find a timed word $\gamma \in \mathcal{L}(N)$ such that $d(\sigma, \gamma) = \min_{x \in \mathcal{L}(N)} d(\sigma, x)$ for some distance function d on timed words.*

In the untimed setting, this is viewed as a problem of minimizing cost over a series of edit moves, either insertions (a model move) or deletions (a trace move). When aligning timed words, clearly there are two crucial aspects to the problem, which are, how one deals with deviations in the action labels themselves (the action labels), versus how one deals with deviations in just the timing properties of the word (the timestamps). The alignment problem has been extensively studied for the untimed case [2] [5], but the timed setting is more complex.

Example 2. *Consider the process model of Example 1 and an observed trace $(reg, 1) \cdot (ec, 2) \cdot (ec, 2) \cdot (ct, 3) \cdot (dec, 8) \cdot (rej, 8)$. Clearly, one shouldn't examine casually twice so the ec is extraneous, and also, the dec event fires too late, it should have waited at most four time units after the firing of event ct. Secondly, consider an observed trace of the form $(reg, 1) \cdot (ec, 2) \cdot (ct, 3) \cdot (dec, 7) \cdot (rej, 8)$. The control flow run itself is valid, but with the timestamps, we notice that the last transition is invalid. We could either edit the rej action to a pay one and leave the timestamp alone, or, edit the timestamp to get $(rej, 7)$. Timestamps are attached to the letters they belong to, so in a sense, firing event rej at time 7 as compared to firing event pay at time 8 are not easily comparable, as the time constraints on different events are often different. In this manner, it is not clear how meaningful it would be to define edit moves that transform timestamps that are attached to different tasks.*

This is why, we first restrict our attention to the case where the untimed part of σ does match the model, that is to say, if $\pi_1(\sigma)$ has a valid causal process (CN, p) over N. Clearly this case of the problem needs to be solved if the general timed alignment problem is to be solved, and we argue that this case is interesting and complex in its own right.

This ensures that we can compare the timing constraints on analogous parts of the process, and hence reason meaningfully about timing deviances from the model. This gives us the following problem :

Definition 13 (The Purely Timed Alignment Problem)**.** *Given a process model N denoted by a TPN and a timed trace w, with timing function $\pi_2(w) = \sigma$ and a valid causal process of the untimed part $\pi_1(w)$, we wish to find a valid timing function γ such that $d(\sigma, \gamma) = \min_{x \in \mathcal{L}(N)} d(\sigma, x)$for some distance function d on timing functions.*

Hence, for sections III, IV and V of this paper, we will ignore the untimed parts of the words, i.e we assume the word σ we wish to align comes with a valid causal process (CN, p) of the model, and seek to find timing functions on said causal process that are both valid, and minimize distance to σ under the metric of choice.

We shall revisit the general timed alignment problem in section VI, and show how our methods can be adapted using existing techniques to provide approaches for solving the general problem.

A. Edit Moves on Timing Functions

Now we come to the problem of deciding how to compare two timing functions over the same causal process, and quantify how close they are to each other.

Much like Levenshtein's edit distance, popularly used in the untimed case of the alignment problem, we view the definition of these distances as an exercise in cost minimisation over the set of all transformations between two words. In order to formalise the same, we need to define what the valid moves of such a transformation could be. We define *moves* as functions that map one timing function over (CN, p) to another. What sort of functions are useful notions of transformation on a timed system?

Example 3. *Let us go back to example 1, and study the process model N presented there. Say we had the following words that did not fit the model, and we wished to analyse how best to modify them to fit them back into the model.*

We start with $\sigma = (reg, 1) \cdot (ct, 2) \cdot (ec, 2) \cdot (dec, 6) \cdot (rej, 6)$. One candidate for the closest valid execution would be $\gamma = (reg, 1) \cdot (ct, 3) \cdot (ec, 2) \cdot (dec, 6) \cdot (rej, 6)$, and it feels reasonable to say that the cost for aligning σ to this γ is 1. A way to arrive at this conclusion is by noticing that on trying to execute σ's firing sequence only the guard for ct fails. If ct were shifted to fire at 3 instead, the whole run would execute without a hitch. This sort of local, almost typographical error can often happen in systems, and it is the simplest kind to fix.

There is, however, another way to look at it. A closest valid timing function would try to preserve the positions of reg and ec but the moment it tries to fire ct, σ runs early. By pushing ct forward, it should take longer for the decision to be made and the rejection to happen, as those events depended on the checking of the ticket being done first. This causes a cascading chain of errors where if ct is moved forward to fire at 3, to preserve the relative relationships between ct and its successors, we get the trace $\gamma' = (reg, 1)\cdot(ct, 3)\cdot(ec, 2)\cdot(dec, 7)\cdot(rej, 7)$, and in this sense, γ' is closer to σ as it preserves more relative gaps. As we just knocked one event a little later by 1 unit, we can view this as a cost 1 edit too. . This is a slightly more complex type of error, but it reflects the real life fact of cascading errors, and hence is an important type of edit to consider when diagnosing such cascading faults.

Based on the above example, we naturally arrive at two types of moves.

We define a *stamp move* as a move that translates the timing function only at a point, i.e., that edits a single element of the timestamp series τ.

Definition 14 (Stamp Move)**.** *Given a timing function $\gamma : E \to \mathbb{R}$, formally, we define this as :*

$\forall x \in \mathbb{R}, e \in E : stamp(\gamma, x, e) = \gamma'$ *where*

$$\forall e' \in E : \gamma'(e') = \begin{cases} \gamma(e') + x & e' = e \\ \gamma(e') & otherwise \end{cases}$$

The next type of move we describe is the more novel and interesting *delay move*. Here, we seek to leverage the structure of the process model itself, by reflecting the causal relationships the pre-order G causes. If an event is G-reachable from another, that means the first event is strictly in the causal history of the second so changing its timestamp should have consequences for all of its causal descendents, while leaving any causally unrelated (non-G-reachable) events undisturbed. Hence, a delay move at e will preserve relative relationships between timestamps in the future, at the cost of shifting the timestamp of every causal descendent of e by the same amount.

Definition 15 (Delay Move)**.** *Given a timing function $\gamma : E \to \mathbb{R}^n$, we define a delay move applied to it as follows :*

$\forall x \in \mathbb{R}, e \in E : delay(\gamma, x, e) = \gamma'$ *where*

$$\forall e' \in E : \gamma'(e') = \begin{cases} \gamma(e') + x & e' \geq_G e \\ \gamma(e') & otherwise \end{cases}$$

The cost of a move is the magnitude $|x|$ in the above definitions, and the cost of a sequence of moves is the sum of the costs of the moves. Armed with these definitions, three natural notions of distance can be constructed.

Definition 16 (Stamp Only Distance : d_t)**.** *Given any two timing functions τ_1, τ_2 over the same causal process (CN, p), we define the stamp-only distance d_t as follows :*

$d_t(\tau_1, \tau_2) = \min\{cost(m) | m \in Stamp^*, m(\tau_1) = \tau_2\}$

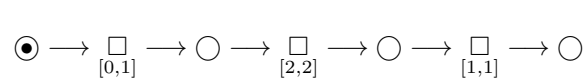
Definition 17 (Delay Only Distance : d_θ)**.** *Given any two timing functions τ_1, τ_2 over the same causal process (CN, p), we define the delay-only distance d_θ as follows :*

$d_\theta(\tau_1, \tau_2) = \min\{cost(m) | m \in Delay^*, m(\tau_1) = \tau_2\}$

Definition 18 (Mixed Moves Distance : d_N)**.** *Given any two timing functions τ_1, τ_2 over the same causal process (CN, p), we define the mixed move distance d_N as follows*

$$d_N(\tau_1, \tau_2) = \min\{cost(m) | m \in (Stamp \cup Delay)^*, m(\tau_1) = \tau_2\}$$

Example 4. *Consider the following example :*



Now for this N, let the observed trace $\sigma = (3, 4, 5) \notin \mathcal{L}(N)$.

The best d_t alignment for the example in the diagram below is $\gamma = (1, 3, 4)$ with minimum cost $d_t(\sigma, \gamma) = 4$.

The best $d_t = \theta$ alignment for the example in the diagram below is also $\gamma = (1, 3, 4)$, but this time with minimum cost $d_\theta(\sigma, \gamma) = 3$, evidenced by the move sequence $(delay(-2, 1)delay(+1, 2))$.

And lastly, the best d_N alignment for the example in the diagram below is also $\gamma = (1, 3, 4)$, the sequence of moves being one stamp and one delay move at the start, $m = stamp(-1, 1)delay(-1, 1)$, and now with minimum cost $d_N(\sigma, \gamma) = 2 < \min\{d_t(\sigma, \gamma), d_\theta(\sigma, \gamma)\}$.

IV. Results and Algorithm : Stamp Edits

Lemma 2 (Stamp only distance : d_t)**.** *d_t as defined above is equivalent to the Manhattan distance or taxicab distance between two points of $\mathbb{R}^n$, where $|E| = n$, say $\mathbf{t} = (t_1, t_2, \ldots t_n)$ and $\mathbf{s} = (s_1, s_2, \ldots s_n)$, defined as*

$$d_t(\mathbf{t}, \mathbf{s}) = \sum_{i=1}^{n} |t_i - s_i|$$

Proof. The way we see this is as follows. Given a causal process $CN = (B, E, G)$ and two timing functions (not necessarily constituting valid timings) τ_1, τ_2, let

$$d_t(\tau_1, \tau_2) = \sum_{e \in E} |\tau_1(e) - \tau_2(e)|$$

This is both achievable by stamp only moves by doing the corresponding $\tau_1(e) - \tau_2(e)$ stamp move to the second word, and is the least cost any such stamp only path can take as Manhattan distance obeys the triangle inequality, and every stamp move results in a word at exactly the same Manhattan distance from its predecessor as the cost of said stamp move. □

This lemma also allows us to restate the alignment problem for d_t, as we see below.

A. Casting Alignment as a Linear Programming Problem

The alignment problem in the stamp-only setting can be viewed as a problem of convex optimization, minimizing the stamp only cost from a fixed observed trace σ, which by Lemma 2 is $cost(\tau) = \sum_{i=1}^{n} |\tau_i - \sigma_i|$ over the set of valid timestamp series τ for the model N. What remains is to show that the space of valid runs of the model is a convex set, as shown in [4].

Hence, we claim that using an efficient encoding of the target function using the firing domain formulation described above, the problem can be cast as a linear programming problem, and LPP solvers can solve this in good time in practice.

Theorem 3. *Given a bounded TPN N, an observed trace σ, and its causal process $CN = (B, E, G)$, the alignment problem can be viewed as seeking the vector $(\gamma_1, \gamma_2, \dots \gamma_{|E|})$ that minimizes the quantity $\sum_{e \in E} |\gamma_e - \sigma_e|$ over the set $\gamma \in \mathcal{L}(N)$ which can be cast as a linear programming problem.*

Proof. Lemma 1 (General Form Lemma) of [4] states that the firing domains of state classes for any bounded TPN may be expressed as solution sets of systems of inequalities of the following form :

$$\begin{cases} a_i \leq t(i) \leq b_i & \forall i \\ t(j) - t(k) \leq c_{jk} & \forall j \neq k \end{cases}$$

As $\gamma \in \mathcal{L}(N)$ this means it obeys the set of inequalities corresponding to the state class containing the final marking. Now, the above constraint is convex, so γ varies over a convex set. As σ is a constant vector, translation by it will keep the space convex, so $\gamma - \sigma \in \mathcal{C}$ for some convex set $\mathcal{C}$.

Now, we wish to minimize $|\gamma - \sigma|$, but it would be better to view this objective as a linear function. To this end we use a trick from [6] and use the following variable translation :

$$\alpha, \beta \geq 0 : \alpha - \beta = \gamma - \sigma$$

As defined above, α and β are the absolute values of the nonnegative and negative components of the vector $\gamma - \sigma$, respectively, so their difference is $\gamma - \sigma$ and their sum is $|\gamma - \sigma|$. Clearly now, we have completed the transformation, and our alignment problem is equivalently cast as

$$\underset{\alpha - \beta \in \mathcal{C}, \alpha, \beta \geq 0}{\text{minimize}} \quad \alpha + \beta$$

□

By the above theorem we see that the alignment problem for d_t is easily cast as a linear progamming problem, and can be solved by any linear programming solver. However this approach does not take into account the specific nature of the problem, and LPP methods can be slow in the worst-case.

This bound is not necessarily tight, but either improving it or proving its tightness are left for future work. While the general case seems to be hard to solve efficiently, there is a subclass of process models for which the problem seems more tractable, and moreover, has a quadratic time solution.

B. A Direct Algorithm for Sequential Models

In this section, we reintroduce a result in [9] in a sligtly different way for the sake of coherence. Given a process model N, an observed trace σ, and its underlying sequential causal process $CN = (B, E, G)$ with homomorphism p for the untimed run of σ on N, we express the cost of aligning a suffix of σ to the corresponding part of a timed trace $\tau \in \mathcal{L}(N)$ on the truncation of the sequential causal process starting from the ith transition, call it $cost(\tau|_{[i,n]})$. By definition, we have that

$$\begin{aligned} cost(\tau|_{[i,n]}) = d_t(\tau|_{[i,n]}, \sigma|_{[i,n]}) &= \sum_{j=i}^{n} |\tau_i - \sigma_i| \\ &= |\tau_i - \sigma_i| + cost(\tau|_{[i+1,n]}) \end{aligned} \tag{1}$$

Observe that aligning $\sigma|_{[i,n]}$ to $\tau|_{[i,n]}$ then can be devided into two steps : First, aligning σ_i with τ_i and then, aligning the remaining suffix $\sigma|_{[i+1,n]}$ to $\tau|_{[i+1,n]}$. Then, for all timing function τ complying with the causal process and $\tau_i = d$, the minimum cost of aligning the suffix $\sigma|_{[i,n]}$ to the corresponding part of $\tau|_{[i,n]}$ is

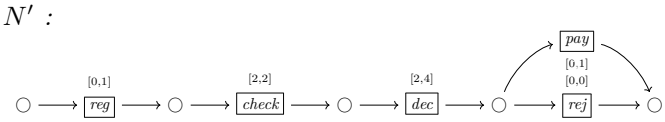
$$\begin{aligned} g_i(d) &= \min_{\substack{\tau \in \mathcal{L}(N) \\ \tau_i = d}} cost(\tau|_{[i,n]}) \\ &= \min_{\substack{\tau \in \mathcal{L}(N) \\ \tau_i = d}} \left[|\tau_i - \sigma_i| + cost(\tau|_{[i+1,n]}) \right] \\ &= |d - \sigma_i| + \min_{\substack{\tau \in \mathcal{L}(N) \\ \tau_i = d \\ \tau_{i+1} - \tau_i \in [a_{i+1}, b_{i+1}]}} cost(\tau|_{[i+1,n]}) \\ &= |d - \sigma_i| + \min_{\substack{\tau \in \mathcal{L}(N) \\ \tau_{i+1} \in [d + a_{i+1}, d + b_{i+1}]}} cost(\tau|_{[i+1,n]}) \\ &= |d - \sigma_i| + \min_{d' \in [d + a_{i+1}, d + b_{i+1}]} \min_{\substack{\tau \in \mathcal{L}(N) \\ \tau_{i+1} = d'}} cost(\tau|_{[i+1,n]}) \\ &= |d - \sigma_i| + \min_{d' \in [d + a_{i+1}, d + b_{i+1}]} g_{i+1}(d') \end{aligned}$$

Thus, for all g function, it follows this recurrent relation :

$$g_i(d) = |d - \sigma_i| + \min_{d' \in [d + a_{i+1}, d + b_{i+1}]} g_{i+1}(d')$$

If we can calculate the family of functions $\forall i \leq n : g_i$ efficiently, then the minimal cost for aligning σ to N is simply $\min_{d \in [a_1, b_1]} g_1(d)$.

Example 5. *We take a moment to look at an example of the computation of this family of functions we proposed by considering d_t-alignment on a simplification of our net from 1, collapsing checking the ticket and the examination steps into one check step : We first have the process model N' :*



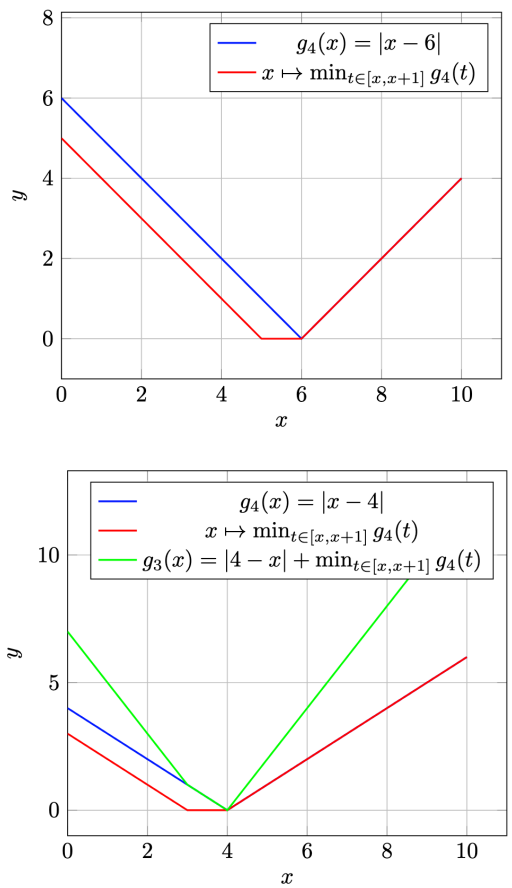


Figure 1. Illustration of a run of the algorithm on Example 5.

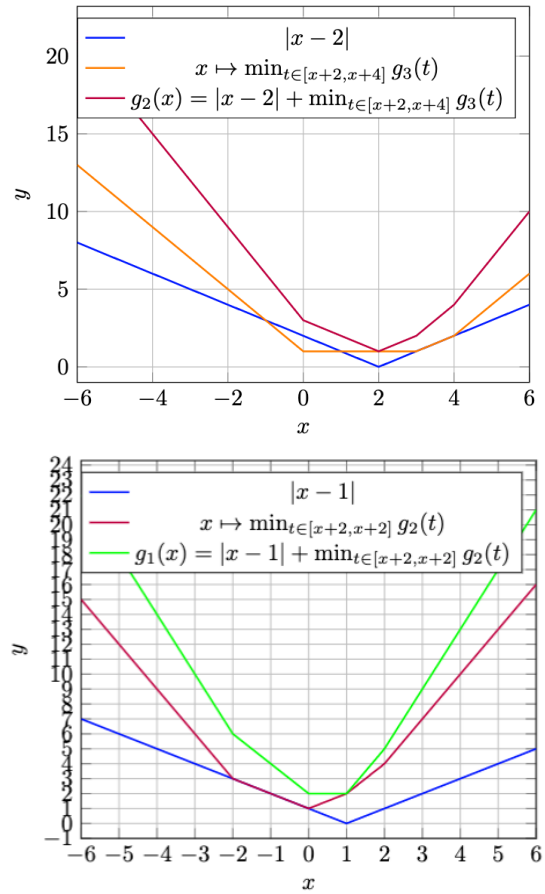


Figure 2. Illustration of a run of the algorithm on Example 5.

The observed trace is $\sigma = (reg, 1) \cdot (check, 2) \cdot (dec, 4) \cdot (pay, 6)$. *We begin to compute the family of functions* g_i, *starting with* $g_4(x) = |6 - x|$. *Utilising the recursion, we proceed to compute* $g_3(x) = |4 - x| + \min_{t\in[x+0,x+1]} g_4(t)$. *Notice that* $[x + 0, x + 1]$ *with* x *increasing represents the translation from left to right of the domain* $[0, 1]$ *and we search for the minimum of* g_4 *in this domain. Since* g_4 *is a V-shaped graph with its lowest point at 6, the minimum of* g_4 *in the domain* $[x + 0, x + 1]$ *is given by the value of* g_4 *at the point in the domain closest to 6, which prouves that*

$$x \mapsto \min_{t\in[x+0,x+1]} g_4(t) = x \mapsto \begin{cases} g_4(x+1) & x \le 5 \\ 0 & 5 < x < 6 \\ g_4(x) & x \ge 6 \end{cases}$$

1. We can then sum this with $|4 - x|$ *to get* g_3.

We construct g_2 *similarly, by summing the contribution of the next timestamp aligning cost,* $|x - 2|$, *to* $\min_{x'\in[x+2,x+4]} g_3(x')$ *to obtain the purple graph in the figure below. By the same reasoning as before, the orange graph of* $\min_{x'\in[x+2,x+4]} g_3(x')$ *is obtained by translating the domain* $[2, 4]$ *from left to right and taking the minimum of* g_3 *in this domain, and then summing it with* $|x-2|$ *gives us the purple graph. We continue this process to get* g_1 *by summing* $|x-1|$ *to* $\min_{x'\in[x+0,x+1]} g_2(x')$, *which is the green graph in the last figure.*

The procedure for constructing g_1 *is identical, and now finally we see that the minimum cost of* 2 *is achieved in* g_1 *by paying at time 0 or 1. Suppose we choose to register at time 1, which makes means that we can deduce the corresponding value of check is* $\operatorname{argmin}_{t\in[1+2,1+2]} g_2(t) = 3$, *and the corresponding value of dec is* $\operatorname{argmin}_{t\in[3+0,3+1]} g_3(t) = 4$, *and by consequence, the corresponding value of pay is* $\operatorname{argmin}_{t\in[4+0,4+1]} g_4(t) = 6$. *We can backtrack in this manner to build an optimal alignment, such as in this case,* $(reg, 1)(check, 3)(dec, 5)(pay, 6)$.

With this context, we present the following algorithm that progressively calculates the graphs of g_i (which are a continuous, piecewise linear, convex function admitting $+\infty$ as limit at $\pm\infty$, represented as lists of segments, each characterised by their slopes and x-projections of endpoints) each round, and claim it is both correct and efficient. The algorithm precisely executes the above procedure, where the subfunction GRAPHMIN takes $g_i(d)$ and outputs

$$\min_{d-d'\in[a,b]} g(d') = \begin{cases} g(d-a) & l + a \le d < m + a \\ g(m) & m + a \le d \le m' + b \\ g(d+b) & m' + b < d \le h + b \end{cases}$$

Input: σ, SI
Output: $cost = \min_{x \in \mathcal{L}(N)} d_t(x, \sigma)$, $\gamma \in \mathcal{L}(N)$ such that $d_t(\gamma, \sigma) = cost$

```
Type: Graph
    left_slope : real
    right_slope : real
    endpoints : list of coordinates
Data: graph_list = list of Graphs, graphmin_list = list of Graphs
Procedure StampOnlyAlgo(σ, SI):
    Initialise System :
        graph_list[n] ← Graph(−1, 1, [(σ[n], 0)])
    while i > 1 do
        [a, b] ← SI[i]
        graphmin_list[i] ← GRAPHMIN(graph_list[i], a, b)
        i ← i − 1
        graph_list[i] ← GRAPHADDMOD(graphmin_list[i], σ[i])
    end
    [a, b] ← SI[1]
    cost ← GRAPHMIN(graph_list[1], a, b)
    γ ← BACKTRACK(σ, graph_list, cost, SI)
    return cost, γ
end
```

Algorithm 1: d_t Algorithm

which is exactly the process of looking for the minimum value of a function over an interval while it translates, and subfunction GRAPHADDMOD takes $\min_{d-d' \in [a,b]} g(d')$ and adds $|d - \sigma_{i-1}|$ to it. The proof of the nature of the functions and how they are constructed will be given in the next section IV-C when we discuss more general cases. Lastly, the BACKTRACK algorithm backtracks through the list of graphs $\{g_i\}_n$ constructed to reverse engineer a word that aligns with the minimal cost calculated. Again, a more detailed algorithm for backtracking will be proposed in the next section IV-C.

Theorem 4. *Algorithm 1 is correct, i.e., it outputs a valid execution $\gamma \in \mathcal{L}(N)$ which minimizes the distance $d_t(\gamma, \sigma)$ to the trace σ.*

Remark 1. *Note that the subfunctions* GRAPHMIN *and* GRAPHADDMOD *both run through the list of endpoints of the graph once each, and hence are linear in the sizes of their inputs, and Algorithm 1 calls each of these subfunctions once for each letter of the observed trace, each time on a graph with size linear in the current suffix. So the cost calculation section of Algorithm 1 has quadratic time complexity in the size of the input.*

The subfunction BACKTRACK *runs a binary search on each stored graph in graphlist to find a trace in the language that does indeed give the minimum cost, letter by letter, and so overall takes $O(n \log(n))$ time, keeping the overall time complexity $O(n^2)$ where $n = |\sigma|$.*

C. Generalisation for Tree-like Models

As discussed previously, we have developped an algorithm to solve the alignment problem for sequential models by building a recurrent link between two consecutive levels. It appears that this approach is really general and can be adapted to more complex models like tree-like structures. Let us first define what we mean by tree-like models.

Definition 19 (Tree-like Time Petri Net)**.** *Let $N = (P,T,F,SI,\Sigma,\lambda,M_0,M_f)$ be a Time Petri Net. N is Tree-like if it's connected, acyclic and*

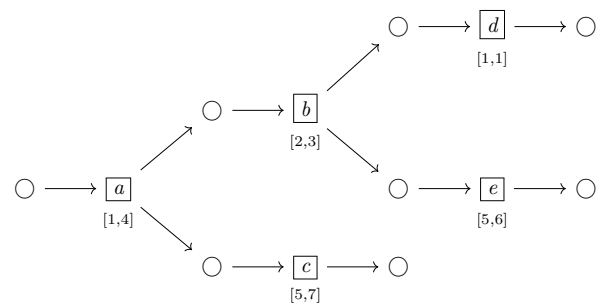
$$\forall t \in T, |t^\bullet| \geq 1 \text{ and } |^\bullet t| = 1,$$

$$\forall p \in P, |p^\bullet| \leq 1 \text{ and } |^\bullet p| \leq 1.$$

N is so called Tree-like Time Petri Net.

Tree-like models are a natural generalisation of sequential models, where activities can branch out into multiple parallel threads of execution and are executed separately since. The fundamental difference of tree-like models comparing to sequential models is its dependencies between higher and lower layers. Precisely, the firing time of an event in a lower layer may affect the firing time of multiple events in the higher layer and by consequence, to fix the firing time of an transition in the lower layer, we may need to take care of multiple events in the higher layer.

Example 6. *We show here an example of Tree-like Time Petri Net and one of the possible traces respecting the process' action order $\sigma = (a,3)(b,10)(c,4)(d,12)(e,15)$.*



Now, the question is how to align σ to the model with minimum cost.

We can firstly try to align σ to the model by greedy approach : From left to right, at each transition, we try to fix its firing time with minimum cost while respecting the static interval constraint. We start by aligning the root of the tree, which is transition a. Since a is executed at 3, which is between 1 and 4, we don't need to change its execution moment.

$$\sigma_{aligned} = (a,3)...$$

Then, we move to a's children : b must be fired between 2 and 3 after a, so the minimal cost of correcting b's

execution moment is 4, by setting it to 6. Similarly, c must be fired between 5 and 7 after a, so we set its execution moment to 8, which costs us 4.

$$\sigma_{aligned} = (a,3)(b,6)(c,8)...$$

Following the same reasoning, d must be fired at 7 and e at 12, which costs us 5 and 3 respectively.

$$\sigma_{aligned} = (a,3)(b,6)(c,8)(d,7)(e,12)$$

Finally, the total cost of aligning σ with the Petri net is 16. Now, by trying to get something better from $\sigma_{aligned}$, we found a better alignment with cost of 15 which is

$$\sigma'_{aligned} = (a,4)(b,7)(c,9)(d,8)(e,13)$$

by remarking that shifting the execution moment of a from 3 to 4 reduces the total cost by 3 (on nodes b, c and d) while increasing the cost by 2 (on a and e).

This example proved that the local optimal choice on a higher layer may not lead to the global optimal solution, we need to consider the effect of changing a node's execution moment on all its descendants. Hence, the advantage of the recurrent relation used in sequential models which computes the alignement cost with respect to the effects on the nodes behind.

Notations and Results

We are given a model N represented as a Tree-like Time Petri Net, an observed trace σ, and its underlying sequential causal process $CN = (B, E, G)$ with homomorphism p for the untimed run of σ on N. As there are no choices for either firing one transition or another in the model, the causal process associated to σ exactly matches the model.

In the following, we focus on the execution moment of every transition and the places don't play any important role here, we will refer to every transition i as *node* and $i^{\bullet\bullet}$ or Child(i) as set of its *children*. Considering the causal process associated to the observed trace σ which is tree-like, we denote by σ_i the execution moment of node i in σ and by $[a_i, b_i]$ its static interval constraint.

Proposition 1. *Let i be a node in the tree-like causal process, and let $g_i(x)$ be the minimum cost of aligning the sub-process rooted at ${}^\bullet i$ when i is fired at time x. Then, $g_i(x)$ obeys the following recurrent relation :*

$$\begin{aligned} g_i(x) &= |\sigma_i - x| + \min\left\{ \sum_{\substack{j \in Child(i) \\ x_j - x \in [a_j, b_j]}} g_j(x_j) \right\} \\ &= |\sigma_i - x| + \min\left\{ \sum_{\substack{j \in Child(i) \\ x_j \in [x+a_j, x+b_j]}} g_j(x_j) \right\} \end{aligned}$$

So, in order to have the minimal alignment cost at node i, the $(x_j)_{j \in \text{Child}(i)}$ must be chosen such that the sum of the alignment costs of all its children is minimized, while respecting their static interval constraints. But there are no constraintes between the choices of x_j, so we can minimize each $g_j(x_j)$ independently. Therefore, we have the following relation :

Proposition 2.

$$\begin{aligned} g_i(x) &= |\sigma_i - x| + \min\left\{ \sum_{\substack{j \in Child(i) \\ x_j \in [x+a_j, x+b_j]}} g_j(x_j) \right\} \\ &= |\sigma_i - x| + \sum_{j \in Child(i)} \min_{x_j \in [x+a_j, x+b_j]} \{g_j(x_j)\} \end{aligned}$$

Now, suppose that we get access to each value of $g_{root}(x)$ as x varies. Then, the minimal cost of aligning the whole sequence with the Petri net is $\min_{x \in [a_i, b_i]}\{g_i(x)\}$, which is exactly the least cost to set the root's execution moment to a value between a_{root} and b_{root} and to align all of its sub-trees.

Proposition 3. *The minimal cost of aligning the sequence with the whole Petri net is*

$$\min_{x \in [a_{root}, b_{root}]} \{g_{root}(x)\}$$

.

We are now interested in the nature of the function $g_i(\cdot)$ to be able to compute it effectively. The two following results give us some key properties of these functions.

Lemma 5. *For any node i of the tree, the function $g_i(\cdot)$ is a continuous, convex, piecewise linear, and diverges to $+\infty$ at $\pm\infty$.*

Proof. We show by induction the property : For every node u of the tree, $H_i : g_i(\cdot)$ is a continuous, convex, piecewise linear function which tends to $+\infty$ at $\pm\infty$. Initially, if i is a leaf, H_i is true directly since $g_{leaf}(x) = |\sigma_{leaf} - x|$ is an absolute value function. We climb up the tree and suppose that for every $j \in \text{Child}(i)$, H_j is true. The two first points above show that

$$\begin{aligned} g_i(x) &= |\sigma_i - x| + \sum_{j \in \text{Child}(i)} \min_{x_j \in [x+a_j, x+b_j]} g_j(x_j) \\ &= |\sigma_i - x| + \sum_{j \in \text{Child}(i)} h_j(x) \end{aligned} \tag{2}$$

where $h_j : x \mapsto \min_{x_j \in [x+a_j, x+b_j]} g_j(x_j)$. By induction hypothesis, $g_j(\cdot)$ is a continuous, convex, piecewise linear function which tends to $+\infty$ at $+\infty$ and $-\infty$. According to Lemma 6 shown further below, h_j also possesses these same properties. Finally, since the sum of continuous, convex, piecewise linear functions which tend to $+\infty$ at $+\infty$ and $-\infty$ is also a function of the same type, we deduce that $g_i(\cdot)$ also possesses these properties. Hence, the result. □

Corollary 1. *Let g a continuous, convex, piecewise linear function whose limits at $+\infty$ and $-\infty$ exist and are $+\infty$. There exist then x_0 and x_1 such that g decreases in $]-\infty, x_0]$, constant in $[x_0, x_1]$ and increases in $[x_1, +\infty[$.*

We refer to these as decreasing, stationary and increasing zones. As a consequence, every function $g_i(\cdot)$ possesses these proprieties.

With these properties in mind, let's pick up the example 6 again to illustrate how to compute these functions in practice.

Example 7. *We consider again the Tree-like Time Petri Net from example 6 and the observed trace $\sigma = (a,3)(b,10)(c,4)(d,12)(e,15)$. We start by computing the functions $g_d(x)$, $g_e(x)$ and $g_c(x)$ associated to the leaves d, e and c. Since they don't have any children, we have :*

$$g_d(x) = |\sigma_d - x| = |12 - x|$$

$$g_e(x) = |\sigma_e - x| = |15 - x|$$

$$g_c(x) = |\sigma_c - x| = |4 - x|$$

Both functions are piecewise linear with a vertex at σ_d, σ_e and σ_c respectively. Then, we move up to node b to compute $g_b(x)$. Using the recurrent relation, we have :

$$g_b(x) = |\sigma_b - x| + \min_{x_d \in [x+1, x+1]} \{g_d(x_d)\} + \min_{x_e \in [x+5, x+6]} \{g_d(x_e)\}$$

To compute $g_b(x)$, we first need to compute the two minimizations. The following lemma will show us how to compute them from the knowledge of $g_d(\cdot)$ and $g_e(\cdot)$.

Lemma 6. *For all real couple a,b s.t $a \leqslant b$, and for all continuous, convex, piecewise linear function f which tends to $+\infty$ at $+\infty$ and $-\infty$,*

$$g : x \mapsto \min_{t \in [x+a, x+b]} f(t)$$

is also a continuous, convex, piecewise linear function having $+\infty$ as limit at $\pm\infty$.

Proof. We present here a method for obtaining the graph of g from the graph of f. A rigorous proof can then be easily deduced.

Now, since f is continuous, convex, piecewise linear, Corollary 1 implies the existence of two real numbers x_0 and x_1 such that f decreases in $]-\infty, x_0]$, constant in $[x_0, x_1]$, increasing in $[x_1, +\infty[$. Then, the minimum of f in the interval $[x+a, x+b]$ that slides from left to right as x increases can be estimated based on the relative position between $[x_0, x_1]$ and $[x+a, x+b]$. We distinguish three cases:

- When the intersection $[x_0, x_1] \cap [x+a, x+b]$ is nonempty, we have

  $$g(x) = \min_{t \in [x+a, x+b]} f(t) = \min_{\mathbb{R}} f,$$

  since the interval $[x+a, x+b]$ contains points where f attains its global minimum. Therefore, the graph of g over the interval $[x_0 - b, x_1 - a]$ is a horizontal segment at height $y = \min_{\mathbb{R}} f$. That is, the graph can be obtained by shifting the graph of f restricted to $[x_0, x_1]$ left by b units and extending it to the right by $(b-a)$ units.
- When $x+b \leq x_0$, the interval $[x+a, x+b]$ lies entirely to the left of x_0, and f decreases in this interval. Hence,

  $$g(x) = \min_{t \in [x+a, x+b]} f(t) = f(x+b).$$

  The graph of g in the interval $]-\infty, x_0 - b]$ is obtained by shifting the graph of f restricted to $]-\infty, x_0]$ left by b units.
- When $x_1 \leq x+a$, the interval $[x+a, x+b]$ lies entirely to the right of x_1, and f increases in this interval. Hence,

  $$g(x) = \min_{t \in [x+a, x+b]} f(t) = f(x+a).$$

  The graph of g in the interval $[x_1 - a, +\infty[$ is obtained by shifting the graph of f restricted to $[x_1, +\infty[$ left by a units.

Finally, g can be drawn correctly as shown in an example illustrated in Figure 3 below.

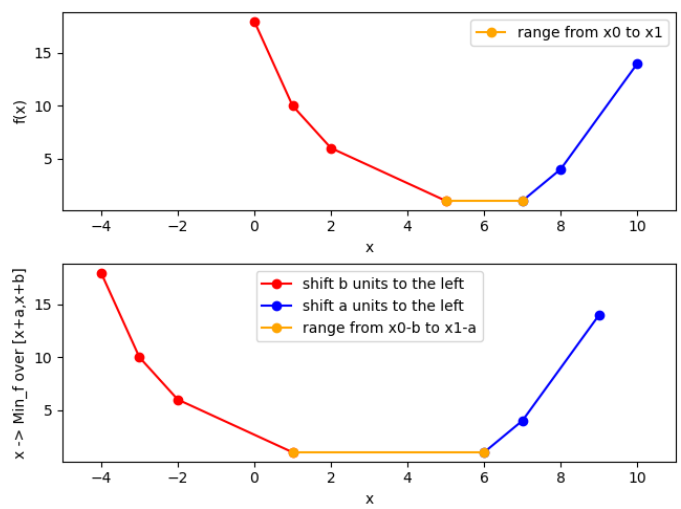


Figure 3. Example with $x_0 = 5$, $x_1 = 7$, a = 1, b = 4

Graphically, it is clear that g is also a continuous, convex, piecewise linear function that has $+\infty$ as limit at $\pm\infty$. A rigorous proof can be driven by formalizing these arguments mentioned above and justifying the convexity of g very easily. □

With all these elements, we can now compute $g_b(x)$ and then $g_a(x)$ without any difficulty. To finish the example 6, we have :

$$g_a(x) = |3 - x| + \min_{x_b \in [2+x, 3+x]} g_b(x_b) + \min_{x_c \in [5+x, 7+x]} g_c(x_c) \tag{3}$$

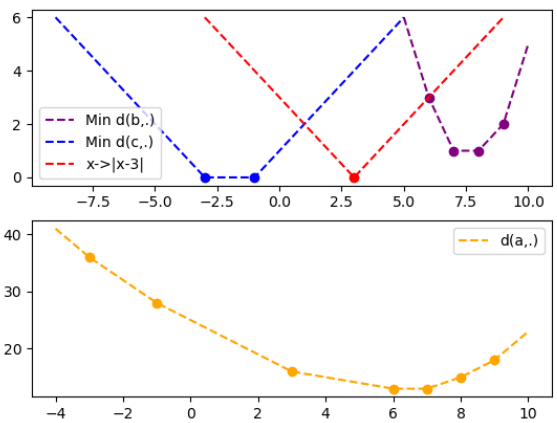


Figure 4. Noting $\forall \tau \in T, g_\tau(\cdot) = d(\tau, \cdot)$, cost function $x \mapsto g_a(x)$

Finally, we obtain the cost function for the root 4. Zooming in a bit, we see that the graph attains its global minima on the interval $[6, 7]$ but it is not the result directly. The first transition is fired between 1 and 4, the minimal cost of our alignment problem is

$$\min_{x \in [1,4]} g_a(x)$$

which is 15, attains at $x = 4$. Looking back to the equation (3), we can deduce that the best alignment is obtained by setting a to 4, then b will be set to x_b which minimizes $g_b(\cdot)$ over $[2 + x, 3 + x]$. Since $x = 4$, $[2 + x, 3 + x] = [6, 7]$ and the minimum of $g_b(\cdot)$ over this interval is attained at 7. So x_b is 7. Similarly, x_c will be set to 9, x_d to 8 and x_e to 13. We then get the aligned trace

$$\sigma'_{aligned} = (a, 4)(b, 7)(c, 9)(d, 8)(e, 13)$$

with a total cost of 15. This is exactly the result we found above.

To sum up the method shown above, the pseudocode 2 of the algorithm is to compute the cost functions $g_i(\cdot)$ for all nodes i in a bottom-up fashion and to find the aligned trace with minimum cost. The following theorem states the correctness of the algorithm.

Theorem 7. *The algorithm 2 computes the optimal alignment of a timed trace σ to a tree-like Time Petri Net N under Stamp Only Distance and the corresponding minimal cost.*

Proof. Before proving the correctness of the algorithm, since the net is tree-like, so confusion-free, there is no need to worry about the condition of urgency. Therefore, three conditions cited in the theorem 1 are reduced to one : The execution moment of each transition must respect its static interval constraint. Now, we prove the correctness of each function used in the algorithm.

- Firstly, we prove the correctness of the function ComputeCostFunc by induction on the height of the tree. Initially, if i is a leaf, the function returns $g_i(x) = |\sigma_i - x|$ which is correct (Line 4). By defining the height of a node its biggest distance to a leaf in its subtree, the inital case was showing the that the function works for nodes of height 0. We climb up the tree and suppose that for every node j of height less than h, the function returns the correct cost function $g_j(\cdot)$. Let i be a node of height h. All children of i have height at most $h-1$, so by induction hypothesis, the function returns the correct cost functions $g_j(\cdot)$ for all $j \in \text{Child}(i)$. Then, according to the recurrent relation shown above, the function computes correctly $g_i(\cdot)$ from these functions (Lines 7-13). So the function works for nodes of height h. Hence, its correctness for all nodes of the tree.
- Now we prove the correctness of the function backTrack once every cost function $g_i(\cdot)$ is computed. According to the recurrent relation shown above

$$g_i(x) = |\sigma_i - x| + \sum_{j \in \text{Child}(i)} \min_{x_j \in [x+a_j, x+b_j]} g_j(x_j)$$

  the execution moment of each node j in the optimal alignment can be computed from the execution moment of its parent (to minimize the cost). By following the structure of Depth-First Search (Lines 16-22), we suppose that the parent of the *root* is executed at 0, then we can compute the execution moment of each node in the optimal alignment correctly. Hence, the correctness of the function backTrack.
- The correctness of Main is then straightforward since it just calls the two previous functions in order (Lines 24-27).

□

To provide some complexity analysis and conclude this section, we note that each function $g_i(\cdot)$ is a continuous, convex, piecewise linear function which can be characterized by its leftmost and rightmost slopes and a set of points located at positions where the slopes change. Also, by summing these functions, the sum is a function of the same type and its slopes change only at points where the slope of one of its components changes. Thus, it is possible to calculate an upper bound for the number of breakpoints of the functions $g_u(\cdot)$ for all node u.

Lemma 8. *For a node u, let $S_{g_u(\cdot)}$ denote the set of breakpoints (points where slope changes) of the function $x \mapsto g_u(x)$, and $S_{\min g_u(\cdot)}$ for $x \mapsto \min_{x_u \in [a_u+x, b_u+x]} g_u(x_u)$. Then:*

1) $|S_{\min g_u(\cdot)}| \leq |S_{g_u(\cdot)}| + 1$
2) $|S_{g_u(\cdot)}| = |\{\sigma_u\} \cup \bigcup_{j \in Child(u)} S_{\min g_j(\cdot)}|$
3) $|S_{\min g_u(\cdot)}| \leq 2|T_u|$ *where T_u is the set of transitions of the subtree whose root is ${}^\bullet u$.*

Proof. Scholium:

1) Based on the proof of Theorem 6, the graph of $x \mapsto \min_{x_u \in [a_u+x, b_u+x]} g_u(x_u)$ can be obtained by translating the graph of $x \mapsto g_u(x)$ and extending its stationary zone. As a result, this extension of the stationary zone can at most add one more point to $S_{\min g_u(\cdot)}$ compared to $S_{g_u(\cdot)}$.
2) We know that

$$g_u(x) = |x - \sigma_u| + \sum_{j \in \text{Child}(u)} \min_{x_j \in [a_j+x, b_j+x]} g_j(x_j)$$

and by adding the affine functions, the slopes add up. Therefore, the sum, which is also a piecewise linear function, can change its slopes only at points where the slope of one of its components changes. Hence the result.
3) We show by induction the property : For every node u of the tree,

$$H_u : |S_{\min g_u(\cdot)}| \leq 2|T_u|.$$

Initially, if u is a leaf, H_u is true directly. We climb up the tree and suppose that for every $j \in \text{Child}(i)$, H_j is true. The two first points above show that

$$\begin{aligned} |S_{\min g_u(\cdot)}| \leq 1 + |S_{g_u(\cdot)}| &= 1 + |\{\sigma_u\} \cup \bigcup_{j \in \text{Child}(u)} S_{\min g_u(\cdot)}| \\ &\leq 1 + |\{\sigma_u\}| + \sum_{j \in \text{Child}} |S_{\min g_u(\cdot)}| \\ &\leq 2 + \sum_{j \in \text{Child}(u)} 2|T_j| \qquad (4) \\ &= 2|T_u| \end{aligned}$$

By deduction, the result is true for all nodes of the tree. Hence, the result.

□

We can now conclude by giving the pseudocode of the algorithm for tree-like models and its complexity 2.

The complexity of the algorithm mainly depends on the calculation of $g_{node}(x)$ for each node of the tree. For a node u, calculating $g_u(x)$ requires summing the contributions of its children, which can be done in $O(|\text{Child}(u)| \cdot |S_{g_j(\cdot)}|)$ time, where $j \in \text{Child}(u)$. Since $|S_{g_j(\cdot)}|$ is bounded by $2|T_j|$ according to the previous lemma, the time complexity for computing $g_u(x)$ becomes $O(|\text{Child}(u)| \cdot |T_j|)$. Summing this over all nodes in the tree, we find that the overall time complexity of the algorithm is $O(|T|^2)$, where $|T|$ is the number of transitions in the Petri net. This quadratic complexity arises from the need to compute and store the cost functions for each node and their children.

Function BACKTRACK traverses the tree once, and at each node, determines the optimal execution moment based on the precomputed cost functions in $O(1)$ time. In total, this function runs in $O(|T|)$ time and so the overall time complexity of the algorithm is $O(|T|^2) + O(|T|) = O(|T|)^2$.

Input: Tree-like Time Petri Net N = (P,T,F,SI,Σ, λ, M_0, M_f) modelized by $G = (V, E)$ where $V = T$ and $(t_i, t_j) \in E$ iff $t_j \in t_i^{\bullet\bullet}$, timed trace $\sigma = (a_1, \sigma_1)(a_2, \sigma_2)...(a_n, \sigma_n)$
Output: $cost = \min_{x \in \mathcal{L}(N)} d_t(x, \sigma)$, $\sigma_{aligned} \in \mathcal{L}(N)$ such that $d_t(\sigma_{aligned}, \sigma) = cost$
Data: $\sigma_{aligned} \leftarrow [], x_{parent} \leftarrow 0$

```
Function ComputeCostFunc(node):
    if g_node is calculated then return g_node ;
    else if node is a leaf then
        return g_node(x) ← absFunc(σ_node)
    end
    else
        g_node = absFunc(σ_node)
        foreach child in Child(node) do
            g_child ← ComputeCostFunc(child)
            min_g_child ←
              graphMin(g_child, a_child, b_child)
            g_node ←
              graphAddMod(g_node, min_g_child)
        end
        return g_node
    end
end
Function backTrack(node, x_parent):
    x_node ←
      argmin_{x_node − x_parent ∈ [a_node, b_node]} g_node(x_node)
    if node is a leaf then
        return
    end
    foreach child in Child(node) do
        backTrack(child, x_node)
    end
end
Function Main():
    root ← root of the tree
    g_root ← ComputeCostFunc(root)
    cost ← min_{x ∈ [a_root, b_root]} g_root(x)
    backTrack(root, 0)
    return cost, σ_aligned
end
```

Algorithm 2: Aligning a timed trace with a Tree-like Time Petri Net under Stamp Only Distance

V. Results and Algorithm : Delay Edits

Delay edits provide a new way of thinking about transformations over time-series, and inspire a new way of viewing timing functions over causal processes. There is of course the standard definition, $\tau : E \to \mathbb{R}^+$ that assigns to each event a timestamp that records exactly when the event occurs.

Instead, thinking along the lines of durations between

events occurring, we will often benefit from considering the following representation when speaking about delay moves, defined as to view a timed word not in terms of its absolute timestamps, but by the delays between them.

Definition 20 (Flow Function)**.** *Given a causal process (CN, p) over N, where $CN = (B, E, G)$, and a (not necessarily valid) timing function $\tau : E \to \mathbb{R}^+$, we first define the flow function of τ, $f_\tau : E \to \mathbb{R}^+$ such that*

$$f_\tau(e) = \begin{cases} \tau(e) & {}^{\bullet\bullet}e = \emptyset \\ \tau(e) - \tau(e') & e' \in {}^{\bullet\bullet}e, \\ & \tau(e') = \max\limits_{e'' \in {}^{\bullet\bullet}e} \{\tau(e'')\} \cup \{0\} \end{cases}$$

Example 8. *Consider again the model in example 1. Say we want to align $w = (reg, 1)(et, 2)(ct, 5)(dec, 6)(rej, 8)$ to N. Consider the values the clock takes during a faulty "execution" $reg \to 1, et \to 1, ct \to 4, dec \to 1, rej \to 2$. This is the flow function.*

Note that e' as defined here is the latest causal predecessor of e . Hence, f_τ as defined produces exactly the time durations that the guards of each transition in the model checks, i.e. the clock function values during the run.

As defined, we see that if a word is in the language of the model, then its f function maps events to values that lie within the event's static interval constraint. This condition is unfortunately not sufficient due to urgency which determines the only firable transition, but it is quite close to the exact condition necessary for a word to be in the language of a time Petri net.

Also note that given the underlying causal process and the resulting f_τ, we can reconstruct τ quite straightforwardly as $\forall e \in E : \tau(e) = \sum_{e' \leq_G e} f_\tau(e')$.

Lemma 9 (Delay only distance : d_θ)**.** *d_θ as defined previously is equivalent to the following formulation directly using the flow function: $d_\theta(\tau_1, \tau_2) = \sum_{e \in E} |f_{\tau_1}(e) - f_{\tau_2}(e)|$.*

Proof. By noting that delay moves on a trace τ translate exactly to stamp moves on f_τ, we can deduce by the same argument as Lemma 2 that the distance function d_θ can be seen to be identical the manhattan distance on flow functions, i.e, the above formulation. □

The flow function is hence a dual representation of timing functions, as if a timing function traditionally labels its transition nodes with timestamps, the flow function labels edges leading up to transitions with the duration since said transition was enabled. This gives us a new way to formulate the alignment problem in terms of minimising distance from the flow vector of σ.

Example 9. *Let us return to example 8. Say we now wish to align w to N. We recall that we calculated its flow function to be*
$reg \to 1, et \to 1, ct \to 4, dec \to 1, rej \to 2$. Pick the closest flow values in the model possible, naively. The resulting flow function is $reg \overset{1}{\mapsto}, et \overset{2}{\mapsto}, ct \overset{2}{\mapsto}, dec \overset{2}{\mapsto}, rej \overset{0}{\mapsto}$ From which we can reconstruct the closest word $u = (reg, 1)(et, 3)(ct, 3)(dec, 5)(rej, 5)$.

Clearly if one tries to move any component of u's flow function any closer to that of w, the word will no longer be in the language.

We could reason this way because d_θ allows one to locally edit traces without affecting membership in the language of the TPN, as both the firing rule and delay edits concern themselves only with the duration of time elapsed between a transition's enabling and its firing. This relationship is especially clear in extended free choice TPNs, where the only events that determine the fireability of an event are its causal history and events with the exact same preset which ensures the interval constraint. Hence, we present Algorithm 3, which minimizes the delay cost at each transition locally, while still overall ensuring membership in the language of the model. The main step in the algorithm is line 12, the rest just ensures validity.

Theorem 10. *Algorithm 3 is correct, i.e., it outputs a valid execution $\gamma \in \mathcal{L}(N)$ which minimizes the distance $d_\theta(\gamma, \sigma)$ to the trace σ.*

Proof. By theorem 1 proved in Aura and Lilius' article [3], we know that in order to ensure that this is a valid time process, we need only ensure three inequalities hold as the time process γ evolves.

1) $\forall e \in E : Eft(p(e)) \leq \gamma(e) - TOE({}^\bullet e, p(e))$
   By definition $f_\gamma(e) = \gamma(e) - TOE({}^\bullet e, p(e))$, and by line 6 of the algorithm the variable eft is correctly initialised to store $Eft(p(e))$, and line 17 ensures the above inequality holds.
2) $\gamma(e) - TOE({}^\bullet e, p(e)) \leq \min\{Lft(t) | {}^\bullet t = {}^\bullet p(e)\}$
   In similar vein, line 6 also ensures that l is initialised precisely to store $\min\{Lft(t) | {}^\bullet t = {}^\bullet p(e)\}$, and line 17 again enforces the left hand side to be less than l.
3) (CN, p) is complete with respect to γ, i.e, for every extension transition t of the process,

$$\max\{\gamma(e) | e \in E\} \leq TOE(Cut(E), t) + Lft(t)$$

   At every point when assigning a timestamp to an event, it is ensured that it belongs to *Soon*, a set defined to contain only those enabled events that have the least value of $l + toe$, or, $TOE(Cut(E), p(e)) + Lft(p(e))$. This ensures that whenever an event e is assigned a timestamp $\gamma(e)$, for all extension transitions t (which must either be causal descendents of e or already enabled when e was picked, thereby being in *Enabled*, by extended free choice) we know that $\gamma(e) \leq TOE(Cut(E), p(e)) + Lft(p(e)) \leq TOE(Cut(E), t) + Lft(t)$.
   Hence, (CN, p, γ) is a valid time process of N.

As for its optimality, we see that any other valid time process would have to obey the same inequalities, in

Input: N, $CN = (B, E, G), p$, $\sigma : E \to \mathbb{R}^+$
Output: $\gamma : E \to \mathbb{R}^+$ such that γ is a valid timing function and $d_\theta(\gamma, \sigma) \leq d_\theta(x, \sigma)$ for all valid x

$Curr = \mathrm{Min}(CN)$
$Enabled = \{t \mid {}^\bullet t \subseteq p(Curr)\}$
$FD = \{(t, \mathrm{Eft}(t), l, 0) \mid t \in Enabled, l = \min_{{}^\bullet t' = {}^\bullet t} \mathrm{Lft}(t')\}$
$Soon = \{(t, eft, l, toe) \in FD \mid l + toe \text{ is minimal in } FD\}$
while $E \neq \emptyset$ **do**
  Pick $(t, eft, l, toe) \in Soon, t \in p(E)$
  $E \leftarrow E \setminus \{e \mid p(e) = t\}$
  $e \leftarrow p^{-1}(t)$
  $Curr \leftarrow \{Curr \setminus \{{}^\bullet e\}\} \cup \{e^\bullet\}$
  $f_\gamma(e) = \mathrm{argmin}_{x \in [eft, l]} |x - f_\sigma(e)|$
  for $t' \in T \wedge {}^\bullet t' = {}^\bullet t$ **do**
    $Enabled \leftarrow Enabled \setminus \{t'\}$
    $FD \leftarrow FD \setminus \{(t', e', l', toe')\}$
  end
  for $t' \in T \wedge t'^\bullet = t^\bullet$ **do**
    $Enabled \leftarrow Enabled \cup \{t'\}$
    $eft' \leftarrow \mathrm{Eft}(t')$
    $l' \leftarrow \min_{{}^\bullet t'' = {}^\bullet t'} \mathrm{Lft}(t'')$
    $toe' \leftarrow toe + f_\gamma(e)$
    $FD \leftarrow FD \cup \{(t', eft', l', toe')\}$
  end
  $Soon = \{(t, eft, l, toe) \in FD \mid l + toe \text{ is minimal in } FD\}$
end
return γ

Algorithm 3: Local d_θ Algorithm

particular for another valid timing function γ' it would have to obey

$$f_{\gamma'}(e) \in [Eft(p(e)), \min_{{}^\bullet t' = {}^\bullet p(e)} Lft(t')] = [a, b]$$

and hence at any event e where γ differs from γ' it would have lower than or equal cost for that event as

$$f_\gamma(e) = \mathrm{argmin}_{x \in [a,b]} |f_\sigma(e) - x|$$

Hence, the algorithm is correct. □

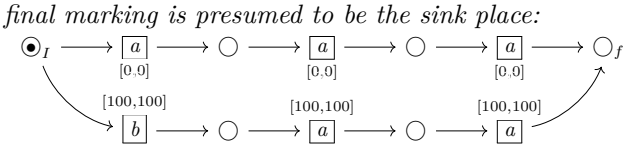
Remark 2. *Note that Algorithm 3 runs through the events of the causal process E exactly once, and each time runs through the set of currently enabled transitions, and the total set of transitions, and hence its time complexity is linear in the size of the input $|CN|$, and the size of the transition set, i.e, $O(|E||T|)$.*

VI. The General Timed Alignment Problem

Now that we have provided methods to tackle the purely timed alignment problem for two metrics, we return to the general timed alignment problem. As our examples throughout have described, the purely timed alignment problem does not so much ignore the action labels as assume that they do not need editing. As a result, one approach for solving the general timed alignment problem would be to begin by completely ignoring the timed aspect, and aligning only the untimed part of the word w and the process model N (now reduced to the underlying untimed Petri net) using the $\mathbb{A}^*$ algorithm [2], which can be used to get a valid control flow, that is, a valid causal process. This, for us, is an untimed word w' which can then be given a timing function based on that of w, repeating or deleting timestamps for any insertion or deletion moves. This yields a valid causal process and a potentially invalid timing function over it, which is then cast as a purely timed alignment problem.

This approach is quite naive, as it may have a tendency to exaggerate the importance of action label errors. For example, consider the following scenario :

Example 10. *Consider the following net N_3 where the final marking is presumed to be the sink place:*



Now, let the observation be $(a, 100)(a, 100)(a, 100)$. The alignment the above approach would provide is $(a, 0)(a, 0)(a, 0)$. But if we assign even a tenth of the cost of action edits to timestamp edits, clearly the closer word in the language was $(b, 100)(a, 100)(a, 100)$.

The issue here is the tradeoff between timestamp edits and action label edits. A better approach would assign one cost c_A to action edits and another c_T to time edits, and seek to minimise their sum. A potential fix to our previous idea would be to essentially retain the above approach, but instead of just doing it for the best untimed alignment, take a selection of good untimed alignments (up to some allowable threshold), and try to align timestamps for each untimed candidate, and then minimise the total cost over this set. This alleviates some of the bias towards preserving action labels by allowing for action deviations in case the timestamp alignment proves particularly easy, but this is still a naive approach to the wider problem, and we hope to find better solutions in the future.

VII. Implementation

We have implemented both the two stamp only alignment algorithms in python, available at https://github.com/NehaRino/TimedAlignments. These algorithms are both quadratic in size of the instances, and runs well in practice. To show that the upper bound complexity is quadratic, we measure the running time of the algorithm for sequential traces of lengths 10, 100, 1000 and 10000 (which are the worst cases) and the measures confirm our analysis. The results are presented in the table and figure below.

Trace Length	10	100	1000	10000
Running Time (seconds)	0.00022	0.018	1.31	182.38

Table I
TREE-LIKE CASES

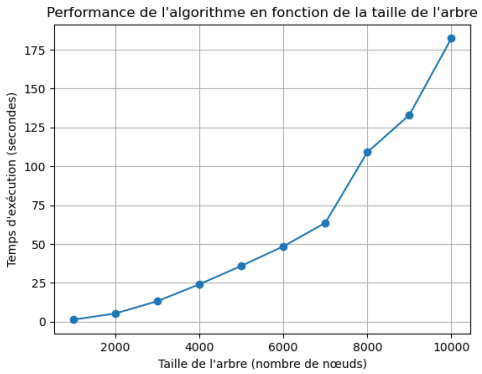


Figure 5. **Tree-like Cases**

VIII. Perspectives and Conclusion

In this paper, we posed the alignment problem for timed processes, devised three metrics for studying alignments for timestamp sequences, and solved the purely timed alignment problem for the first two metrics proposed. As far as we know, this is the first step in conformance checking for time-aware process mining, and much further work can be inspired from this point. The alignment problem for the third metric d_N is a first future direction. Secondly, for both metrics studied here the class of models for which the alignment problem was solved efficiently are structurally restricted (being tree-like causal processes for d_t and extended free choice time Petri nets for d_θ) and it would be interesting to see how to broaden their scope to larger classes of process models. Thirdly, further investigation in the general timed alignment problem is necessary, as our proposed approach here is rather rudimentary and can certainly be improved. Lastly, there are a number of other conformance artefacts that can be set and studied in the timed setting, such as anti-alignments [8], and one can better develop all such conformance checking methods to account for timed process models.

References

[1] Wil Aalst, Helen Schonenberg, and Minseok Song. Time prediction based on process mining. *Inf. Syst.*, 36:450–475, 04 2011.

[2] Arya Adriansyah. *Aligning observed and modeled behavior*. PhD thesis, Technische Universiteit Eindhoven, 2014.

[3] Tuomas Aura and Johan Lilius. Time processes for time petri-nets. In Pierre Azéma and Gianfranco Balbo, editors, *Application and Theory of Petri Nets 1997, 18th International Conference, ICATPN '97, Proceedings*, volume 1248 of *LNCS*, pages 136–155. Springer, 1997.

[4] Bernard Berthomieu and Maurice Menasche. A state enumeration approach for analyzing time petri nets. In *Application and Theory of Petri Nets*, Varenna, Italy, 1982.

[5] Mathilde Boltenhagen, Thomas Chatain, and Josep Carmona. A discounted cost function for fast alignments of business processes. In *BPM 2021, Proceedings*, volume 12875 of *LNCS*, pages 252–269. Springer, 2021.

[6] Stephen P. Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2014.

[7] Josep Carmona, Boudewijn F. van Dongen, Andreas Solti, and Matthias Weidlich. *Conformance Checking - Relating Processes and Models*. Springer, 2018.

[8] Thomas Chatain, Mathilde Boltenhagen, and Josep Carmona. Anti-alignments - measuring the precision of process models and event logs. *Inf. Syst.*, 98:101708, 2021.

[9] Thomas Chatain and Neha Rino. Timed alignments. In *4th International Conference on Process Mining, ICPM 2022*, pages 112–119. IEEE, 2022.

[10] Saoussen Cheikhrouhou, Slim Kallel, Nawal Guermouche, and Mohamed Jmaiel. The temporal perspective in business process modeling: a survey and research challenges. *Service Oriented Computing and Applications*, 9:75–85, 2014.

[11] Raffaele Conforti, Marcello La Rosa, Arthur H. M. ter Hofstede, and Adriano Augusto. Automatic repair of same-timestamp errors in business process event logs. In *BPM 2020, Proceedings*, volume 12168 of *LNCS*, pages 327–345. Springer, 2020.

[12] Johann Eder, Euthimios Panagos, and Michael Rabinovich. Time constraints in workflow systems. In *CAiSE'99, Proceedings*, volume 1626 of *Lecture Notes in Computer Science*, pages 286–300. Springer, 1999.

[13] An Nguyen, Srijeet Chatterjee, Sven Weinzierl, Leo Schwinn, Martin Matzner, and Bjoern M. Eskofier. Time matters: Time-aware lstms for predictive business process monitoring. In Sander J. J. Leemans and Henrik Leopold, editors, *Process Mining Workshops - ICPM 2020 International Workshops, Padua, Italy, October 5-8, 2020, Revised Selected Papers*, volume 406 of *Lecture Notes in Business Information Processing*, pages 112–123. Springer, 2020.

[14] Andreas Rogge-Solti, Ronny Mans, Wil M. P. van der Aalst, and Mathias Weske. Repairing event logs using timed process models. In *On the Move to Meaningful Internet Systems: OTM 2013, Proceedings*, volume 8186 of *LNCS*, pages 705–708. Springer, 2013.

[15] Wil M. P. van der Aalst. *Process Mining - Data Science in Action, Second Edition*. Springer, 2016.

[16] Wil M. P. van der Aalst and Luis F. R. Santos. May I take your order? - on the interplay between time and order in process mining. In *BPM 2021 International Workshops*, volume 436 of *Lecture Notes in Business Information Processing*, pages 99–110. Springer, 2021.